VectorEditGym: Evaluating Specification-Faithful SVG Repair Under Preservation Constraints

Yug Aditi Gupta* **Prannay Hebbar***
Theta Labs
yug@thetalab.tech prannay@warping.co
*Equal contribution; the paper and accompanying material are shared work.

Abstract

Instruction-based vector editing requires two capabilities: making a requested change and leaving everything else alone. The second is easy to miss when an output is judged only as a raster image. We introduce **VectorEditGym**, a compact, difficult benchmark of 40 SVG repair tasks. Each task pairs a corrupted SVG program with a human-authored visual instruction, a hidden target program, 5.05 annotated repairs on average, and an average of 60.55 protected objects. Instructions describe visible defects without exposing element identifiers, coordinates, color codes, or path data. We define a deterministic binary specification reward: requested repairs use attribute-aware perceptual tolerances, while unrequested rendering-relevant structure must remain semantically unchanged and the result must be a valid SVG. Canonical target equality and stricter source fidelity are retained as diagnostics. Continuous repair progress, a near-complete tier, and Unintended Change Rate (UCR) explain partial outcomes. We evaluate 34 model endpoints (25 listed as open-weight, 5 inexpensive controls, and 4 frontier closed endpoints) over 1360 requests. The strongest endpoint reaches only 15.0% full specification success, despite 50.6% mean repair progress, showing that apparent repair progress and specification-faithful editing remain substantially different. All prompts, outputs, scoring code, costs, and per-task reports are released.

1 Introduction

An instruction such as "the amber signal has gone dark; fix it and leave the station alone" defines both a positive requirement and a large set of negative requirements. The signal must change, while the train, clock, cables, platform, and every unrelated program detail must not. A model can produce a plausible raster while renaming elements, rewriting path data, shifting an unrelated object, or deleting metadata. These are not cosmetic differences for an SVG: they alter an executable, structured artifact (World Wide Web Consortium, 2018).

Most generation metrics emphasize final appearance. That is appropriate for open-ended synthesis, but insufficient for local editing. A repair benchmark must answer three separate questions: Did the requested objects change? Did unmentioned objects remain stable? Is the resulting program valid and usable? VectorEditGym makes all three observable through hidden target SVGs and explicit protected-object sets.

The benchmark is deliberately small and dense rather than broad and shallow. Its 40 tasks contain 202 annotated repairs across station, harbor, campsite, market, and larger scenic illustrations. The public instruction never names a DOM identifier or serialized numeric/path value. Solvers must interpret the visible scene and edit the source accordingly. Figure 1 shows representative inputs and targets.

Our contributions are:

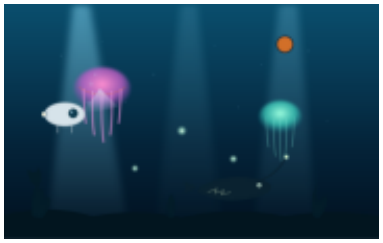
1. a frozen corpus of dense, human-described SVG repairs with explicit intended edits and broad preservation surfaces;
2. a transparent executable evaluator whose binary specification reward combines tolerant requested-edit checks, rendering-relevant semantic preservation, and SVG validity, while retaining source fidelity separately;
3. a reproducible 34-endpoint evaluation with response provenance, token usage, cost, failures, and model-produced SVGs; and
4. an empirical account of the gap between completing visible repairs and preserving the full vector program.

Corrupted input

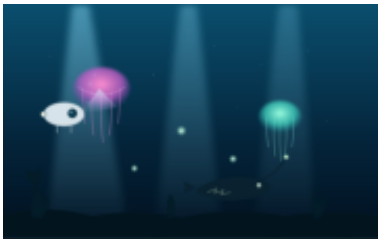
Hidden target

The amber signal has gone dark, the train door is sitting too far to the right, and the overhead cable has an awkward sag. Put those three details back in place and leave the rest of the station alone.

Corrupted input

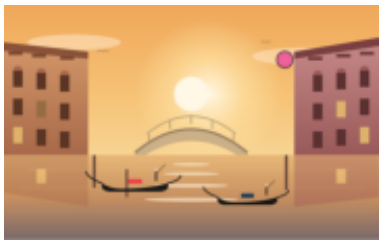


Hidden target

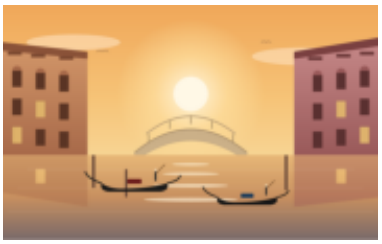


In the deep-sea scene, remove the stray colored marker and move the displaced glowing particle back toward the nearby cluster. The large jellyfish should have fine tentacles and a soft pink lower rim, while the second shaft of light should be clearly visible. Leave the other creatures alone.

Corrupted input



Hidden target



Remove the colored marker from the Venetian sunset. Center the large sun glow behind the canal, restore the dark red body of the gondola, slim the main arch of the bridge, and bring back the warm glow in the affected window. Keep the other buildings and reflections intact.

Figure 1: Representative corrupted inputs and hidden targets. Public prompts describe visible mistakes in ordinary language without naming source IDs or revealing target numeric/path values.

2 Related Work

Vector generation. IconShop generates vector icons from text (Wu et al., 2023); StarVector generates SVG code from images and text (Rodriguez et al., 2025); and LLM4SVG studies complex vector understanding and generation with language models (Xing et al., 2025). These systems establish SVG as a useful meeting point between visual generation and program synthesis. VectorEditGym focuses on local repair rather than open-ended generation.

Instruction-based SVG editing. SVGEdit-Bench introduced quantitative evaluation for LLM-based SVG editing (Nishina and Matsui, 2024), and SVGEditBench V2 expanded instruction-based editing evaluation (Nishina and Matsui, 2025). VectorEdits provides a larger dataset and benchmark for vector-graphics editing (Kuchař et al., 2025). Our scope is complementary: we emphasize dense multi-repair scenes, tolerance-aware repair checks, semantic preservation outside the requested fields, and publication of every model output. The hidden target anchors deterministic checks without requiring a model to reproduce every requested coordinate or color exactly.

3 Benchmark

3.1 Task Format

Each task is a tuple

$$\tau = (x, S_0, S^*, E, U),$$

where x is a human repair instruction, S_0 is the corrupted SVG, S^* is the hidden target, E is a set of expected object-attribute repairs, and U is the set of object IDs that must be preserved. The model receives only (x, S_0). Target source, expected diffs, target-part lists, and preserve lists are excluded from the prompt and recorded as hidden evaluator inputs.

Instructions were rewritten after task freezing by the authors. They describe visible objects and relations—for example, a moon that drifted down and left or a jellyfish whose tentacles became too heavy—without SVG IDs, numeric coordinates, hex colors, path commands, or attribute names. A corpus regression hash covers every non-instruction task field, ensuring this language-only rewrite did not change inputs, targets, annotations, or ordering.

Statistic	Value
Tasks	40
Annotated repairs	202
Repairs per task (mean)	5.05
Protected objects per task (mean)	60.55
SVG elements per task (mean)	85.4
Input characters (total)	294,798

Table 1: Frozen corpus statistics. Element counts include non-rendering SVG nodes; protected-object counts refer to identified scene objects.

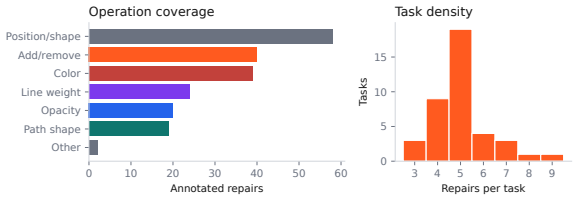


Figure 2: Repair operations and per-task edit density.

3.2 Composition and Difficulty

The corpus contains 20 compact but busy scenes derived from four authored scene templates and 20 larger scenic illustrations. Ten tasks are marked hard and 30 very hard. Tasks require between three and eight repairs, including insertion or deletion, recoloring, displacement, path restoration, line-weight correction, and opacity restoration. All unrequested identified parts are protected.

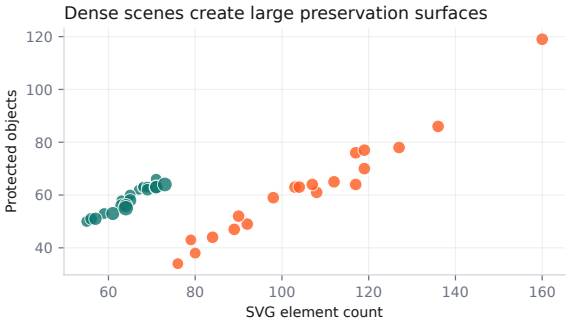


Figure 3: SVG size and preservation surface. Marker area scales with the number of requested repairs.

4 Executable Evaluation

4.1 Three-Gate Specification Reward

Let $C(S)$ parse an SVG and canonicalize representation-only variation. We discard namespace spelling differences and attribute order, normalize numeric spellings and separators in paths, points, transforms, and view boxes, and map common equivalent color spellings (for example, `white`, `#fff`, and `rgb(255,255,255)`) to one form. A second form $V(S)$ represents rendering-relevant semantics: consistently renamed IDs and their local references are aligned by tree position, and equivalent inline-style and presentation-attribute storage is merged. Element order, nesting, geometry, paint, text, and unresolved references remain strict. Malformed XML has neither form. Requested objects are resolved by ID first and by stable tree position when an otherwise equivalent editor has renamed IDs.

For each requested edit $e \in E$, an attribute-aware predicate $\mathrm{ok}_e(S)$ compares the produced value with the hidden target. Existence and non-visual values are exact. Scalar geometry uses absolute error; colors use CIE Lab ΔE_{76}; and point lists use coordinate RMS. Paths are sampled at 129 arc-length positions and compared with a symmetric nearest-point distance combining RMS and the 95th percentile. This admits the same visible curve expressed with different SVG commands. For matching command streams, coordinate RMS remains an additional acceptance route. When the corrupted-to-target distance is δ_e, the tolerance is bounded by

$$t_e = \min(c_e, \max(f_e, r_e\delta_e), 0.9\delta_e),$$

so the known corrupted value cannot pass. Scalar checks use $r_e = 0.50$, color uses 0.45, point lists use 0.55, and paths use 0.70. Position caps are 2.5% of the relevant viewport dimension, color is capped at 18 ΔE_{76}, opacity at 0.12, and path/point geometry at 3% of the smaller viewport dimension. Stroke width is capped at one user unit or 40% of target width, whichever is larger. Floors avoid rejecting immaterial numeric noise. Every per-check report exposes target distance, tolerance, baseline corruption distance, and progress.

Approximation is permitted only at requested fields. Let M_E mask those fields in both the produced and target trees (and remove requested insertion/deletion sites). The primary preservation gate requires $V(M_E(S)) = V(M_E(S^*))$. It accepts consistent ID renaming and style-storage rewrites but catches changes to unlisted definitions, anonymous nodes, root attributes, element order, and unrequested visual attributes. We separately report the stricter source predicate $C(M_E(S)) = C(M_E(S^*))$. The validity gate requires a complete SVG root, well-formed XML, unique nonempty IDs, and syntactically valid normalized attributes.

The primary reward remains binary:

$$\begin{aligned} R_{\mathrm{spec}}(S) = {} & \mathbb{1}[\mathrm{valid}(S)] \\ & \cdot \mathbb{1}[\forall e \in E : \mathrm{ok}_e(S)] \\ & \cdot \mathbb{1}[V(M_E(S)) = V(M_E(S^*))]. \end{aligned}$$

Canonical target equality $\mathbb{1}[C(S) = C(S^*)]$ and source preservation are diagnostics and are not required for a pass. No learned judge or model-based grader appears in the public protocol.

4.2 Evaluator Audit

The evaluator is tested as an executable artifact rather than treated as an unexamined source of labels. The target SVG must receive reward one; the corrupted input must fail at least one requested repair; and unintended deletions, unrequested attribute changes, duplicate identifiers, broken references, malformed paths, and unknown added elements must fail the preservation or validity gate. Regression tests also cover approximate numeric and color repairs, alternate path command topologies, equivalent style storage, and consistent identifier renaming. The frozen corpus has a regression hash over every non-instruction task field. These checks establish implementation invariants, not human perceptual validity: the comparator's scope and the absence of a human agreement study remain limitations.

4.3 Diagnostic Metrics

For expected checks E, tolerance-aware edit completion is

$$\mathrm{Edit}(S) = \frac{1}{|E|} \sum_{e \in E} \mathbb{1}[\mathrm{ok}_e(S)].$$

For checks with a distance, repair progress is $q_e = \mathrm{clip}(1 - d_e/\delta_e, 0, 1)$; exact and existence checks use binary progress. We report $|E|^{-1} \sum_e q_e$ in addition to completion. A *near-complete* output is valid and semantically preserved, misses at most one requested check, and has at least 80% mean progress. It still receives zero binary reward.

For protected objects U, preservation and unintended-change rate are

$$\mathrm{Pres}(S) = \frac{1}{|U|} \sum_{u \in U} \mathbb{1}[V(u_S) = V(u_{S^*})],$$
$$\mathrm{UCR}(S) = 1 - \mathrm{Pres}(S).$$

We additionally report all-repairs pass rate, whole-document semantic-clean rate, source-preservation rate, SVG validity, source exact match, canonical target match, provider errors, scheduled elapsed time, tokens, and cost. Model-level UCR is averaged over valid outputs so truncation is not mislabeled as collateral editing. A provider error or invalid output receives reward zero.

5 Model Study

5.1 Protocol

We evaluate 25 endpoints designated as open-weight in the study manifest, 5 inexpensive closed controls, and 4 frontier closed endpoints through the OpenRouter chat-completions API (OpenRouter, 2026). These designations record the endpoint grouping used for analysis and are not an independent license audit. The exact endpoint IDs appear in Appendix B.

Every model receives the same system instruction, human repair request, and corrupted SVG source. We record one scored outcome per task and omit temperature rather than imposing a provider-incompatible value. In this run, input-adaptive output ceilings range from 4,096 to 7,002 tokens. The runner retries rate limits and transient server failures against the same endpoint. It never falls back to another model; the client permits two transport retries inside each harness attempt. We record requested and resolved model IDs, response ID, finish reason, prompt, completion and reasoning tokens when supplied, scheduled wall-clock elapsed time, and provider-reported cost. Elapsed time begins when a model–task job is scheduled and therefore includes local concurrency-queue wait; we expose it for run auditing, not as endpoint latency. Each source cohort uses a reservation-based budget guard capped at $25; the completed combined study cost $14.88.

This is an endpoint comparison, not a general ranking of model intelligence. It uses one decoded response per model–task pair and does not include a human or oracle-editing baseline. The released artifact supports exact reruns of the scored protocol, but does not estimate sampling variance, human performance, or performance on unseen SVG distributions.

5.2 Uncertainty

We compute 95% confidence intervals by resampling the 40 tasks with replacement 10,000 times independently for each model. These intervals quantify task-set uncertainty only. They do not estimate sampling variance because each model–task pair has one scored outcome and no repeated decoding. Endpoint tables are ordered by full specification pass, mean repair progress, lower valid-output UCR, validity, and model name.

6 Results

Specification success remains rare. Only 32 of 1360 outcomes satisfy all three gates (2.35%), while 14 (1.03%) are near-complete and 682 (50.1%) make at least one annotated repair. Mean repair progress is 35.3%. Across all outcomes, 3.24% satisfy every requested repair and 19.85% preserve the masked document semantically; the stricter source gate passes 19.78%, and only 0.07% canonically match the full hidden target. The strongest endpoint, Claude Sonnet 5, obtains 15.0% specification success. Its repair progress is substantially higher at 50.6%, while its valid-output UCR is 0.8% and SVG validity is 62.5%. The difference is the central benchmark result: models frequently perform some requested visual work but fail the full repair-and-preserve contract. Figure 4 reports task-bootstrap intervals for every endpoint.

Frontier endpoints remain subject to the same bottlenecks. Within the frontier cohort, Claude Sonnet 5 ranks highest with 15.0% specification

Model	Full ↑	Near ↑	Repair ↑	Progress ↑	Clean ↑	UCR ↓	Valid ↑	Cost
Claude Sonnet 5	15.0	2.5	15.0	50.6	42.5	0.8	62.5	$2.032
KAT Coder Air V2.5	12.5	0.0	15.0	38.2	25.0	0.3	42.5	$0.128
MiniMax M3	10.0	0.0	10.0	42.2	25.0	1.0	45.0	$0.240
HY 3	7.5	2.5	10.0	56.8	37.5	1.8	100.0	$0.092
Gemma 4 31B IT	5.0	5.0	10.0	57.3	47.5	0.9	90.0	$0.084
Qwen3.5 397B A17B	5.0	0.0	5.0	46.7	32.5	1.4	65.0	$0.618
DeepSeek V4 Pro	5.0	2.5	5.0	38.7	20.0	1.2	47.5	$0.508
Kimi K2.6	5.0	5.0	5.0	29.2	15.0	0.3	25.0	$1.077
Qwen3 Coder	2.5	0.0	5.0	54.7	45.0	1.4	100.0	$0.295
Gemini 3.1 Flash Lite	2.5	0.0	7.5	54.5	25.0	2.5	100.0	$0.269

Table 2: Top ten endpoints under binary specification reward. Full, Near, Repair, Progress, Clean, UCR, and Valid are percentages; UCR is conditional on valid outputs, and cost is provider-reported for all 40 tasks. Full results are in Appendix B.

success and 50.6% repair progress. Its SVG validity is 62.5% and its truncation rate is 40.0%. Length-limited outcomes elsewhere in the cohort remain failures because the protocol requires a complete usable artifact.

Repair and preservation are separate axes. Figure 5 (left) plots mean repair progress against valid-output UCR. A model can move right by correcting visible defects while also moving upward by rewriting protected scene objects. Reporting completion alone would hide this behavior; reporting only full success would hide useful partial capability.

Operation families expose different bottlenecks. Figure 6 decomposes the expected checks. Simple color or add/remove corrections need not predict path-shape or position restoration, especially because the prompt gives visual descriptions instead of literal DOM values. This decomposition also distinguishes an endpoint that attempted an edit but selected the wrong source element from one that returned an invalid or unchanged document.

Cost is not a sufficient capability proxy. The run spans inexpensive small endpoints and larger reasoning or coding models. Figure 5 (right) shows repair progress against total 40-task cost. It is a descriptive endpoint comparison rather than a hardware-efficiency claim: OpenRouter pricing and routing can change, and the study uses one scored outcome per task.

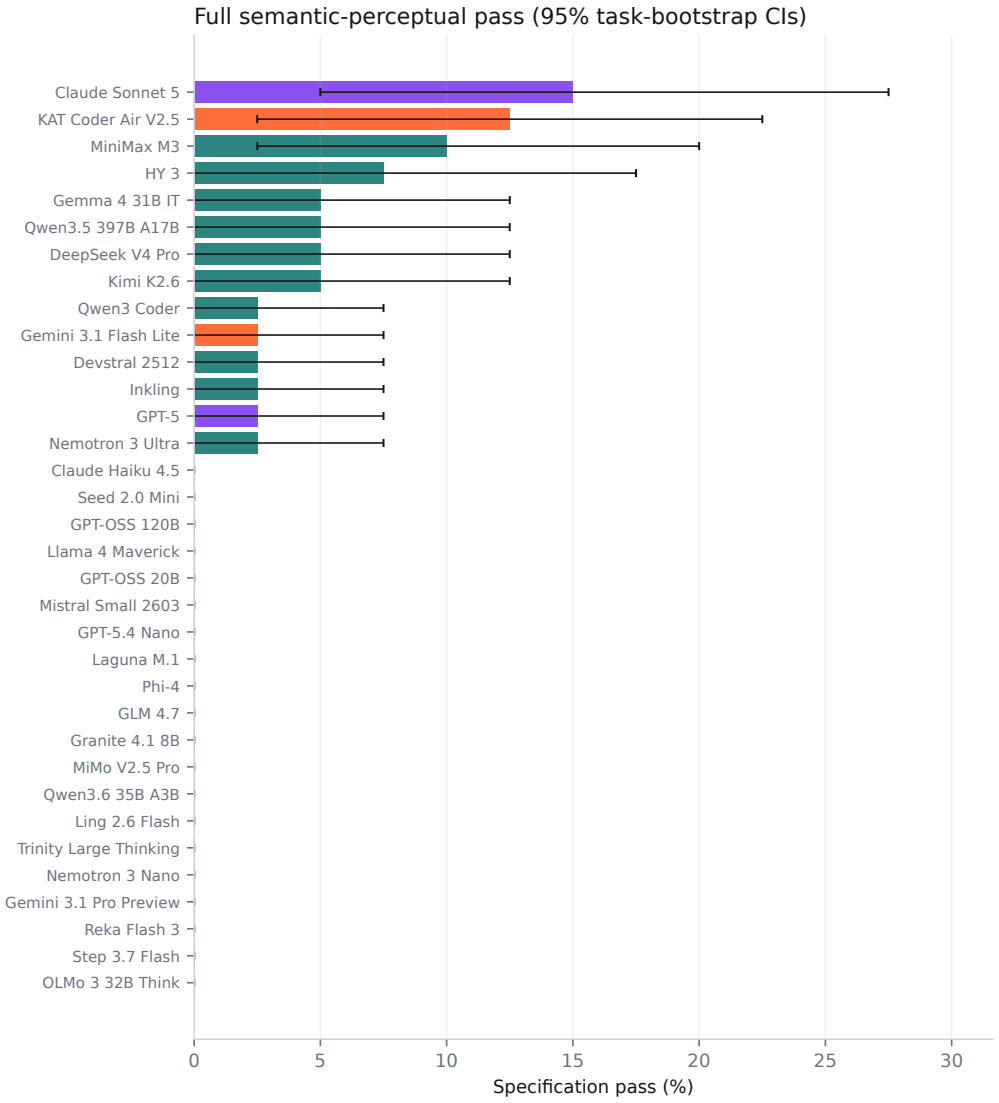
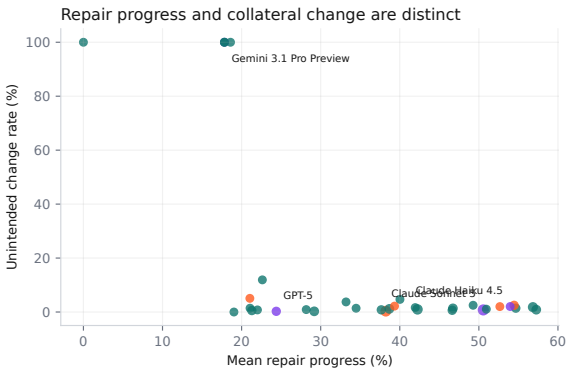
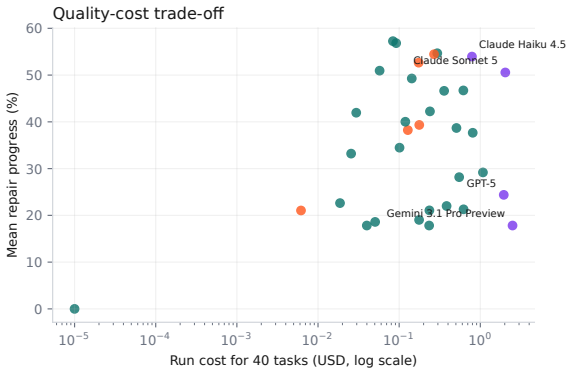


Figure 4: Binary specification success for all endpoints. Orange bars are inexpensive controls, teal bars are endpoints listed as open-weight, and purple bars are frontier closed endpoints.



(a) Repair progress versus unintended change.



(b) Repair progress versus provider-reported cost.

Figure 5: Complementary model-level diagnostics. In (a), marker area scales with specification pass rate and lower right is preferable. Cost in (b) is the recorded total for 40 tasks on a logarithmic axis.

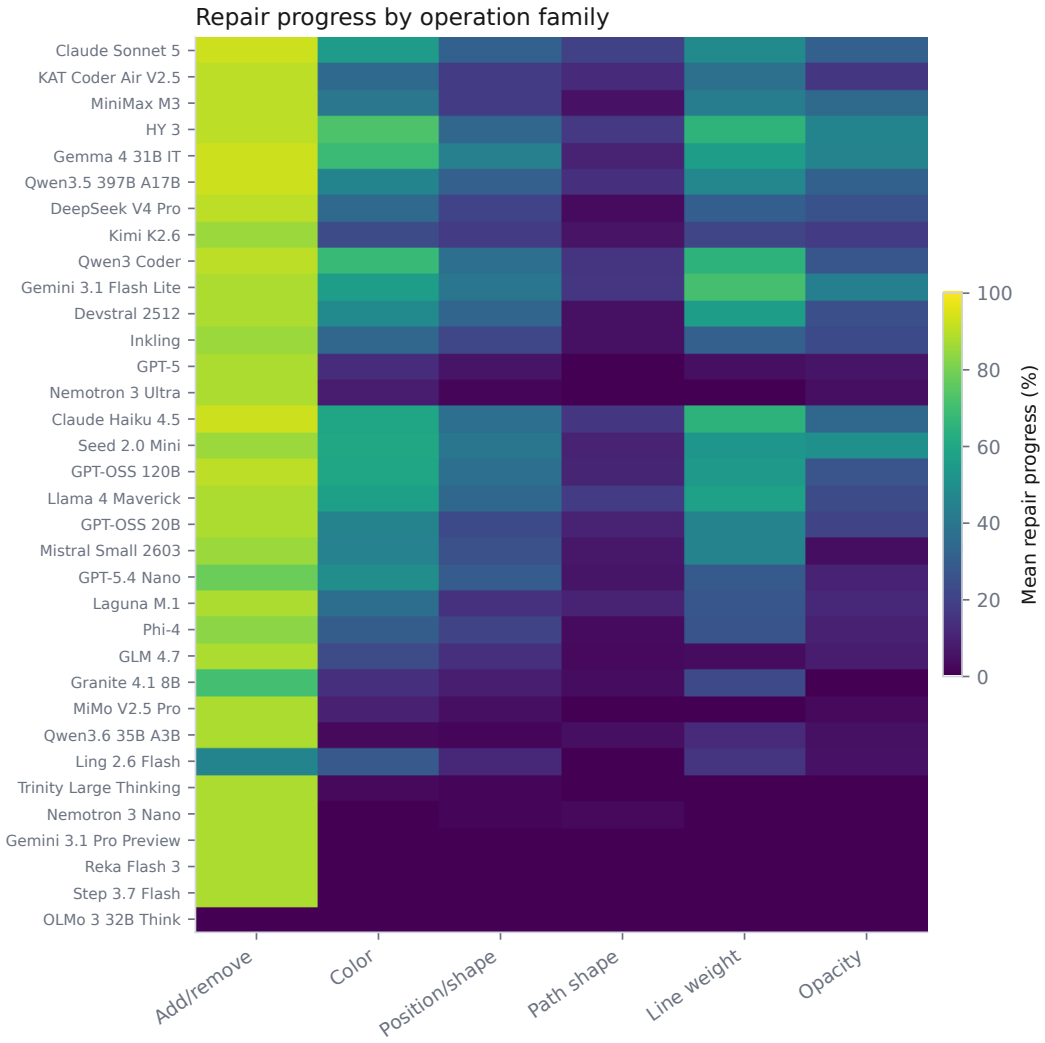


Figure 6: Mean expected-edit repair progress by operation family. Rows follow overall specification-pass rank.

7 Error Analysis

Literal partial repair. Models often identify an obvious semantic defect, such as a signal with the wrong color, while leaving a displaced object or malformed curve unresolved. These outputs earn partial edit-completion credit but specification reward zero.

Global rewrites. Some responses regenerate or reformat substantial portions of the SVG. The semantic normalizer accepts harmless formatting, equivalent style storage, and consistent ID renaming, but changed visible paths, reordered scene objects, broken references, or missing definitions count as changes. The strict source-fidelity diagnostic still surfaces source rewrites when their visual semantics are preserved.

Approximate repair without collateral change. Prompts intentionally omit exact coordinates, color codes, and path commands. The tolerance-aware repair gate accepts bounded alternatives to the hidden target, but only when the remainder of the SVG is unchanged. This separates a reasonable visual approximation from a global rewrite. The full-target match diagnostic quantifies canonical reproduction without controlling the primary reward.

Provider and truncation failures. Only 54.2% of scored outcomes pass structural SVG validity. The harness preserves raw malformed output, API errors, and finish reasons in the released records. Transient failures are retried only against the requested endpoint; persistent failures remain zero-scored observations. The aggregate provider error rate is 3.0%, and 37.8% of all outcomes end at the provider's output-length limit; parse failure is measured separately.

8 Limitations and Artifact Risks

VectorEditGym has only 40 tasks and should be treated as a focused stress test, not a comprehensive measure of visual editing. Public targets make future contamination possible. The deterministic tolerances are transparent but cannot represent every perceptually acceptable alternative. Path comparison uses finite geometric samples and can miss narrow or highly localized differences; CIE76 is only an approximate color metric, and viewport-relative caps may be too strict or permissive for particular scenes. The semantic comparator does not execute arbitrary CSS, browser layout, animation, scripting, accessibility, or editor-specific behavior.

The evaluation uses one scored outcome per model–task pair and therefore cannot characterize sampling stability. OpenRouter can change prices, routing, model versions, and availability; requested and resolved IDs are published to make this visible. The "open-weight" grouping is manifest metadata, not a legal determination.

Twenty scenic source files in the frozen corpus lack recoverable per-file provenance metadata. We do not claim original authorship of those illustrations. They are retained to preserve the evaluated task set, marked as provenance pending in the artifact, and should be treated as research-only until provenance is resolved. This limits redistribution confidence and is a material artifact risk.

Finally, the benchmark evaluates text-and-source editing rather than multimodal screen interaction. Models see SVG code and a visual description, but not a raster preview or interactive editor. Performance therefore combines scene reasoning, source localization, and code generation under one protocol.

9 Conclusion

VectorEditGym isolates a practical requirement for editing agents: requested change is not enough; preservation is part of the specification. Human visual instructions make the task less like copying a patch, while executable SVG targets make every decision auditable. Across 34 endpoints, partial repair is much more common than specification-faithful repair. The released outputs and diff reports make that gap inspectable at the object and source level and provide a compact target for improving structured visual editors.

A Artifact Notes

Reproducibility. The release includes the frozen corpus, prompts, runner, evaluator, tests, analysis, paper source, Harbor tasks, and per-output viewer. Credentials and transport-level API envelopes are excluded.

Shared authorship. Yug Aditi Gupta and Prannay Hebbar contributed equally to the benchmark, artifact, analysis, and writing. The paper and accompanying material are shared work.

B Full Endpoint Results

Values are percentages. Errors include persistent provider failures and missing usable SVG output. Endpoint IDs, resolved IDs, response IDs, and per-task details are available in the machine-readable release.

Model	Group	Full	Near	Repair	Progress	Clean	UCR	Valid	Trunc.	Errors
Claude Sonnet 5	frontier	15.0	2.5	15.0	50.6	42.5	0.8	62.5	40.0	0.0
KAT Coder Air V2.5	cheap-control	12.5	0.0	15.0	38.2	25.0	0.3	42.5	57.5	0.0
MiniMax M3	open-weight	10.0	0.0	10.0	42.2	25.0	1.0	45.0	55.0	0.0
HY 3	open-weight	7.5	2.5	10.0	56.8	37.5	1.8	100.0	0.0	0.0
Gemma 4 31B IT	open-weight	5.0	5.0	10.0	57.3	47.5	0.9	90.0	5.0	0.0
Qwen3.5 397B A17B	open-weight	5.0	0.0	5.0	46.7	32.5	1.4	65.0	32.5	0.0
DeepSeek V4 Pro	open-weight	5.0	2.5	5.0	38.7	20.0	1.2	47.5	52.5	0.0
Kimi K2.6	open-weight	5.0	5.0	5.0	29.2	15.0	0.3	25.0	72.5	2.5
Qwen3 Coder	open-weight	2.5	0.0	5.0	54.7	45.0	1.4	100.0	0.0	0.0
Gemini 3.1 Flash Lite	cheap-control	2.5	0.0	7.5	54.5	25.0	2.5	100.0	0.0	0.0
Devstral 2512	open-weight	2.5	5.0	2.5	46.6	57.5	0.6	97.5	0.0	0.0
Inkling	open-weight	2.5	0.0	2.5	37.7	27.5	0.8	45.0	55.0	0.0
GPT-5	frontier	2.5	0.0	2.5	24.4	7.5	0.3	15.0	85.0	0.0
Nemotron 3 Ultra	open-weight	2.5	0.0	2.5	21.3	5.0	0.5	7.5	92.5	0.0
Claude Haiku 4.5	frontier	0.0	0.0	2.5	54.0	27.5	2.0	100.0	0.0	0.0
Seed 2.0 Mini	cheap-control	0.0	2.5	2.5	52.7	35.0	2.0	92.5	2.5	0.0
GPT-OSS 120B	open-weight	0.0	5.0	0.0	50.9	42.5	1.1	95.0	5.0	0.0
Llama 4 Maverick	open-weight	0.0	0.0	2.5	49.3	20.0	2.5	92.5	0.0	0.0
GPT-OSS 20B	open-weight	0.0	2.5	2.5	42.0	27.5	1.6	72.5	27.5	0.0
Mistral Small 2603	open-weight	0.0	0.0	0.0	40.0	5.0	4.7	95.0	0.0	0.0
GPT-5.4 Nano	cheap-control	0.0	0.0	0.0	39.3	22.5	2.2	97.5	0.0	0.0
Laguna M.1	open-weight	0.0	2.5	0.0	34.5	25.0	1.4	42.5	55.0	0.0
Phi-4	open-weight	0.0	0.0	0.0	33.2	12.5	3.7	95.0	0.0	0.0
GLM 4.7	open-weight	0.0	0.0	0.0	28.2	12.5	0.9	30.0	70.0	0.0
Granite 4.1 8B	open-weight	0.0	0.0	0.0	22.6	10.0	11.9	77.5	2.5	0.0
MiMo V2.5 Pro	open-weight	0.0	0.0	0.0	22.0	5.0	0.8	10.0	87.5	0.0
Qwen3.6 35B A3B	open-weight	0.0	0.0	2.5	21.1	2.5	1.4	7.5	92.5	0.0
Ling 2.6 Flash	cheap-control	0.0	0.0	0.0	21.0	15.0	5.1	90.0	5.0	0.0
Trinity Large Thinking	open-weight	0.0	0.0	0.0	19.0	0.0	0.0	2.5	97.5	0.0
Nemotron 3 Nano	open-weight	0.0	0.0	0.0	18.6	0.0	100.0	0.0	80.0	0.0
Gemini 3.1 Pro Preview	frontier	0.0	0.0	0.0	17.8	0.0	100.0	0.0	100.0	0.0
Reka Flash 3	open-weight	0.0	0.0	0.0	17.8	0.0	100.0	0.0	12.5	0.0
Step 3.7 Flash	open-weight	0.0	0.0	0.0	17.8	0.0	100.0	0.0	100.0	0.0
OLMo 3 32B Think	open-weight	0.0	0.0	0.0	0.0	0.0	100.0	0.0	0.0	100.0

References

Josef Kuchař, Marek Kadlčík, Michal Spiegel, and Michal Štefánik. 2025. VectorEdits: A dataset and benchmark for instruction-based editing of vector graphics. *Preprint*, arXiv:2506.15903.

Kunato Nishina and Yusuke Matsui. 2024. SVGEdit-Bench: A benchmark dataset for quantitative assessment of LLM's SVG editing capabilities. *Preprint*, arXiv:2404.13710.

Kunato Nishina and Yusuke Matsui. 2025. SVGEditBench V2: A benchmark for instruction-based SVG editing. *Preprint*, arXiv:2502.19453.

OpenRouter. 2026. Api reference and model catalog. `https://openrouter.ai/docs/api/reference/overview`. Accessed 2026-07-20.

Juan A. Rodriguez, Abhay Puri, Shubham Agarwal, Issam H. Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli. 2025. StarVector: Generating scalable vector graphics code from images and text. *Preprint*, arXiv:2312.11556.

World Wide Web Consortium. 2018. Scalable vector graphics (SVG) 2. Technical report, W3C.

Ronghuan Wu, Wanchao Su, Kede Ma, and Jing Liao. 2023. IconShop: Text-guided vector icon synthesis with autoregressive transformers. *Preprint*, arXiv:2304.14400.

Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu, and Qian Yu. 2025. Empowering LLMs to understand and generate complex vector graphics. *Preprint*, arXiv:2412.11102.